

OpenSDK 开发指南

修订历史 (Revision history)

编号	修订内容描述	修订人	修订日期	修订后版本号	批准人
1	文档优化	陶波	2025-03-10	1.1.3	
2	升级 sdk 版本, 文档未改动	陶波	2025-06-19	1.2.0	
3	增加卡录像下载	陶波	2025-11-18	1.3.0	
4	增加 arc 设备对讲	陶波	2026-01-27	1.4.0	
5	增加 x86 版本	陶波	2026-04-02	1.5.0	
6	增加 p2p 和云录像功能	陶波	2026-04-13	3.2.0	
7	适配低功耗设备	陶波	2026-05-26	3.3.0	
8	解决 p2p 解密截断的问题	陶波	2026-07-03	3.3.1	



目录

OpenSDK 开发指南	1
目录	2
1 概述	7
2 开发环境配置	8
2.1 SDK 包产物	8
2.2 添加 SDK 包	8
2.3 Visual studio 工程设置	8
3 SDK 接口说明	10
3.1 概述	10
3.2 LCOpenSDK_Define	10
3.2.1 参数相关枚举	10
3.2.1.1 E_STREAM_TYPE	10
3.2.1.2 E_MEDIA_FORMAT	10
3.2.1.3 E_MEDIA_TYPE	11
3.2.1.4 E_LOG_LEVEL	11
3.2.1.5 E_DEVICE_STATE	12
3.2.2 回调相关枚举	12
3.2.2.1 E_RESULT_PROTO_TYPE	12
3.2.2.2 E_RESULT_PROTO_TYPE_RTSP_STATE	13
3.2.2.3 E_RESULT_PROTO_TYPE_DHHTTP_STATE	14
3.2.2.4 E_RESULT_PROTO_TYPE_FILE_STATE	16
3.3 LCOpenSDK_VideoPlay	17
3.3.1 initVideoPlay	17
3.3.2 uninitVideoPlay	17
3.3.3 getIndex	17
3.3.4 setIndex	17
3.3.5 setDisplayRegion	18
3.3.6 setPlayerListener	18
3.3.7 getPlayerListener	18
3.3.8 playAudio	18

3.3.9	stopAudio	19
3.3.10	seek.....	19
3.3.11	pause.....	19
3.3.12	resume	19
3.3.13	rewake	19
3.3.14	snapShot	20
3.3.15	startRecord	20
3.3.16	stopRecord.....	20
3.3.17	setStreamCallback	21
3.3.18	doScale.....	21
3.3.19	getScale.....	21
3.3.20	translate.....	21
3.3.21	getTranslateX.....	22
3.3.22	getTranslateY	22
3.3.23	setMaxScale	22
3.3.24	setIdentity.....	22
3.3.25	getCurrentStreamMode	23
3.3.26	playRtspReal	23
3.3.27	playDeviceRecord	23
3.3.28	playCloud.....	24
3.3.29	stopDeviceRecord	25
3.3.30	stopCloud.....	25
3.3.31	stopRtspReal	26
3.3.32	setSEnhanceMode	26
3.3.33	isRecording	26
3.3.34	isStreamPlayed	26
3.3.35	getCurrentStreamMode	27
3.3.36	getPlayerStatus.....	27
3.3.37	setKey.....	28
3.3.38	setDecodeEngine	28

3.4	LCOpenSDK_AudioTalk.....	28
3.4.1	setTalkerListener	28
3.4.2	getTalkerListener	29
3.4.3	playTalk.....	29
3.4.4	stopTalk	29
3.4.5	playSound	30
3.4.6	stopSound.....	30
3.4.7	startSampleAudio	30
3.4.8	stopSampleAudio.....	30
3.5	LCOpenSDK_Utils	31
3.5.1	getSaveStreamPath	31
3.5.2	computeMD5Lower.....	31
3.5.3	computeMD5Upper.....	31
3.5.4	hmac_sha512.....	32
3.5.5	sha512Cry	32
3.5.6	compute1.....	32
3.5.7	compute2.....	33
3.5.8	getMac.....	33
3.5.9	getOsVersion	33
3.5.10	loadSubModules	33
3.5.11	setMobileLogLevel.....	34
3.5.12	setSaveStreamFlag.....	34
3.6	LCOpenSDK_PlayerListener(播放业务及窗口事件回调)	34
3.6.1	onPlayerResult.....	35
3.6.2	onResolutionChanged	35
3.6.3	onReceiveData.....	35
3.6.4	on PlayBegan	35
3.6.5	onPlayFinished.....	36
3.6.6	onPlayerTime.....	36
3.6.7	onBadFile	36

3.6.8	onFrameLost.....	37
3.6.9	onRecordStop.....	37
3.6.10	onFileTime.....	37
3.6.11	onPlayerTimeAndStamp.....	37
3.6.12	onAudioTimeAndStamp.....	38
3.6.13	onStreamLogInfo.....	38
3.7	LCOpenSDK_TalkerListener.....	38
3.7.1	onTalkPlayReady.....	38
3.7.2	onTalkResult.....	39
3.8	LCOpenSDK_Download.....	39
3.8.1	startDownload.....	39
3.8.2	startCloudDownload.....	40
3.8.3	stopDownload.....	40
3.9	LCOpenSDK_DownloadListener.....	41
3.9.1	onDownloadReceiveData.....	41
3.9.2	onDownloadState.....	41
3.9.3	onDownloadProgress.....	41
4	SDK 对接使用说明.....	41
4.1	SDK 对接流程和关键业务接口调用.....	42
4.1.1	SDK 对接流程.....	42
4.1.2	关键业务接口调用.....	43
4.1.2.1	实时预览.....	43
4.1.2.2	高标清切换.....	44
4.1.2.3	本地录制.....	44
4.1.2.4	抓图.....	44
4.1.2.5	云台控制.....	44
4.1.2.6	语音对讲.....	45
4.1.2.7	设备录像回放.....	46
4.1.2.8	设备录像下载.....	47
4.1.2.9	ARC 对讲.....	48
4.1.2.10	启用 p2p.....	48
4.1.2.11	低功耗设备.....	48
4.2	Demo 功能演示说明.....	49
4.2.1	背景.....	49

4.2.2	功能介绍和使用说明	49
4.2.2.1	账号登录	49
4.2.2.2	获取设备列表	50
4.2.2.3	实时预览和对讲	51
4.2.2.4	云台控制	51
4.2.2.5	设备录像播放	51
4.2.2.6	设备录像下载	52
4.2.2.7	云录像播放	53
4.2.2.8	云录像下载	54



1 概述

本文档主要从以下几方面对如何集成 LCOpenSDK 进行说明，以 Windows 为例：

开发环境配置	介绍在 Visual Studio 中如何将 PC SDK 添加到工程项目中去，通过 Demo 搭建的方式进行演示。
SDK 接口说明	介绍 LCOpenSDK 各个接口的功能、参数、返回值以及注意事项。
SDK 对接使用说明	介绍 SDK 对接的流程、关键业务接口调用以及 demo 的演示说明。



2 开发环境配置

2.1 SDK 包产物

产物	接口	说明
LCOpenSDKmd.dll	提供一系列静态库	视频播放控制类相关业务（各类预览、各类回放、录制、抓图、拖动、暂停恢复、缩放、平移、鱼眼等）
		语音对讲业务
		平台交互业务
		直播业务
		加解密
		以上业务相关头文件
		根据应用协议业务的不同，按需装载。
OpenSDK /include	头文件目录	
其他依赖库	LCOpenApiClientmd.dll 等	协议库

库说明

库名	是否必选	说明
LCOpenSDKmd. dll	必选	SDK 总体库, 实现通用的基础的组件能力, 可动态装载业务库

2.2 添加 SDK 包

需将动态 dll 依赖添加至工程。

2.3 Visual studio 工程设置

- 1) 路径：文件->打开->项目/解决方案

-
- 2) 添加 Header:项目属性->配置->配置属性->C/C++ ->常规->附加包含目录,
在其中添加头文件目录
 - 3) 添加 Library: 项目属性->配置->配置属性->链接器->常规->附加库目录,
在其中添加库文件目录



3 SDK 接口说明

3.1 概述

开放平台 SDK 以动态库(包括.h 文件和.dll 或.lib 文件)的形式发布，提供多个方法类：
LCOpenSDK_VideoPlay（视频播放类），LCOpenSDK_AudioTalk（语音对讲类），LCOpenSDK_Utils（公用方法类），应用只需集成上述方法类，即可快速实现视频能力集成。

此外，提供 LCOpenSDK_PlayerListener、LCOpenSDK_TalkerListener 监听接口类，分别配合 LCOpenSDK_VideoPlay、LCOpenSDK_AudioTalk 使用，应用可继承并设置相应的监听接口类来接收播放、对讲过程中的业务回调，业务回调包括但不限于：播放窗口事件、视频播放过程中的状态码、语音对讲过程中的状态码、解码时间戳，回调接口请参考后续说明。

3.2 LCOpenSDK_Define

3.2.1 参数相关枚举

3.2.1.1 E_STREAM_TYPE

方法 E_STREAM_TYPE		
简介	码流类型枚举	
值	STREAM_TYPE_MAIN	主码流
	STREAM_TYPE_MINOR_1	辅码流 1
	STREAM_TYPE_MINOR_2	辅码流 2
	STREAM_TYPE_MINOR_3	辅码流 3
备注		

3.2.1.2 E_MEDIA_FORMAT

方法 E_MEDIA_FORMAT

简介	视频格式枚举	
值	E_MEDIA_FORMAT_MEDIA_DAV	DAV 格式
	E_MEDIA_FORMAT_MEDIA_MP4	MP4 格式
	E_MEDIA_FORMAT_MEDIA_AVI	AVI 格式
	E_MEDIA_FORMAT_MEDIA_ASF	ASF 格式
	E_MEDIA_FORMAT_MEDIA_FLV	FLV 格式
	E_MEDIA_FORMAT_MEDIA_MOV	MOV 格式
	E_MEDIA_FORMAT_MEDIA_MP464	MP464 格式
	E_MEDIA_FORMAT_MEDIA_MOV64	MOV64 格式
	E_MEDIA_FORMAT_MEDIA_MP4NOSEEK	MP4NOSEEK 格式
	E_MEDIA_FORMAT_MEDIA_WAV	WAV 格式
备注		

3.2.1.3 E_MEDIA_TYPE

方法	E_MEDIA_TYPE	
简介	媒体类型	
值	E_MEDIA_TYPE_VIDEO	视频
	E_MEDIA_TYPE_AUDIO	音频
	E_MEDIA_TYPE_ALL	视频+音频
备注		

3.2.1.4 E_LOG_LEVEL

方法	E_LOG_LEVEL	
简介	日志等级	
值	E_LOG_LEVEL_FAULT	致命错误
	E_LOG_LEVEL_ERR	错误
	E_LOG_LEVEL_WARNING	可能导致出错
	E_LOG_LEVEL_INFO	当前运行状态

	E_LOG_LEVEL_DEBUG	详细调试信息
	E_LOG_LEVEL_ALL	所有日志信息
备注		

3.2.1.5 E_DEVICE_STATE

方法 E_DEVICE_STATE		
简介	设备状态	
值	E_DEVICE_STATE_ON_LINE	在线
	E_DEVICE_STATE_OFF_LINE	离线
	E_DEVICE_STATE_UNKNOWN	未知
备注		

3.2.2 回调相关枚举

3.2.2.1 E_RESULT_PROTO_TYPE

方法 E_RESULT_PROTO_TYPE		
简介	onPlayerResult 回调 type 参数枚举(业务类型)	
值	E_RESULT_PROTO_TYPE_RTSP	RTSP 业务
	E_RESULT_PROTO_TYPE_HLS	HLS 业务
	E_RESULT_PROTO_TYPE_FILE	FILE 业务
	E_RESULT_PROTO_TYPE_NETSDK	NETSDK 业务
	E_RESULT_PROTO_TYPE_SIP	SIP 业务
	E_RESULT_PROTO_TYPE_DHHTTP	HTTP 优化拉流业务
	E_RESULT_PROTO_TYPE_DPSDK	DPSDK 业务
	E_RESULT_PROTO_TYPE_Express	Express 业务
	E_RESULT_PROTO_TYPE_CloudBase	视频云业务
	E_RESULT_PROTO_TYPE_RTP	RTP 业务
	E_RESULT_PROTO_TYPE_FINANCECLOUD	招行金融云业务

	E_RESULT_PROTO_TYPE_COMMUNITYCLOUD	社区云业务
	E_RESULT_PROTO_TYPE_MSP	MSP 业务
	E_RESULT_PROTO_TYPE_P2P	P2P 业务
	E_RESULT_PROTO_TYPE_REST	REST 业务
备注	<i>RTSP 业务(包括实时预览、本地录像回放、对讲)</i> <i>HLS 业务(包括云录像播放、云录像下载)</i> <i>FILE 业务(包括本地文件播放)</i> <i>NETSDK 业务(包括P2P 设备实时预览、录像回放)</i> <i>SIP 业务</i> <i>REST 业务(涉及以上 RTSP、HLS、SIP 业务)</i> <i>DPSDK、Express、CloudBase 为行业平台业务</i>	

3.2.2.2 E_RESULT_PROTO_TYPE_RTSP_STATE

方 法 E_RESULT_PROTO_TYPE_RTSP		
简介	E_RESULT_PROTO_TYPE_RTSP_STATE 业务对应 code	
值	E_RESULT_PROTO_TYPE_RTSP_STATE_PACKET_COMPONENT_ERROR	组件内调用流媒体接口失败
	E_RESULT_PROTO_TYPE_RTSP_STATE_PACKET_FRAME_ERROR	组帧失败
	E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_T_EARDOWN_ERROR	内部要求关闭,如连接断开等
	E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_DESCRIBE_READY	会话已经收到 Describe 响应
	E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_AUTHORIZATION_FAIL	RTSP 鉴权失败
	E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_PLAY_READY	收到 PLAY 响应

	E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_FI LE_PLAY_OVER E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_P AUSE_READY E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_K EY_MISMATCH E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_LI VE_PAUSE_ENABLE E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_LI VE_PAUSE_DISABLE E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_T ALK_BUSY_LINE E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_T ALK_CHECK_FAILED E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_S ERVICE_UNAVAILABLE E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_U SER_INFO_BASE_START E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_S TREAM_LIMIT_NOTIFY E_RESULT_PROTO_TYPE_RTSP_STATE_RTSP_C ONCURRENT_LIMIT_NOTIFY	录像文件回放正常结束 收到 PAUSE 响应 密钥不正确 服务端直播支持 PAUSE 服务端直播不支持 PAUSE 对讲忙线 对讲不满足操作条件 连接最大数错误(基于 503) 用户信息起始码 流量限制通知 并发限制通知
备注		

3.2.2.3 E_RESULT_PROTO_TYPE_DHHTTP_STATE

方法	E_RESULT_PROTO_TYPE_DHHTTP
简	E_RESULT_PROTO_TYPE_DHHTTP_STATE 业务对应 code

介		
值	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_COMPONENT_ERROR	-1, 组件内调用流媒体接口失败
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_OK	1000, 开启播放成功
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_PLAY_FILE_OVER	2000, 回放时, 当前文件播放完毕
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_BAD_REQUEST	400000, 非法请求, 关闭客户端
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_UNAUTHORIZED	401000, 未授权, 用户名密码错
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_FORBIDDEN	403000, 禁止访问, 关闭客户端
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_NOTFOUND	404000, 未找到, 关闭客户端
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_REQ_TIMEOUT	408000 请求超时, 指拉流成功, 但后续网络异常, 导致拉流断开
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_SERVER_ERROR	500000, 服务器内部错误, 关闭客户端

	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_SERVER_UNVALILABLE	503000, 服务不可用
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_FLOWLIMIT	503006, mts 限流
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_P2P_MAXCONNECT	503007, p2p 达到最大链接数
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_GATEWAY_TIMEOUT	504000, 网络不通
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_CLIENT_ERROR	1000000, 客户端内部错误, 一般代码逻辑错误
	E_RESULT_PROTO_TYPE_DHHTTP_STATE_DHH TTP_KEY_ERROR	1000005, 客户端密钥和服务 端密钥不一致
备 注		

3.2.2.4 E_RESULT_PROTO_TYPE_FILE_STATE

方法	E_RESULT_PROTO_TYPE_FILE	
简介	E_RESULT_PROTO_TYPE_FILE 业务对应 code	
值	E_RESULT_PROTO_TYPE_FILE_INIT_FAILED	-1, 本地文件播放初始 化失败
	E_RESULT_PROTO_TYPE_FILE_SEEK_FAILED	6, 拖拽失败
	E_RESULT_PROTO_TYPE_FILE_SEEK_SUCCESS	7, 拖拽成功

	E_RESULT_PROTO_TYPE_FILE_CROSS_BORDER	8, 拖拽越界
备注		

3.3 LCOpenSDK_VideoPlay

3.3.1 initVideoPlay

方法	int initVideoPlay(int index, void* hWnd);	
简介	初始化播放窗口	
参数	index	播放窗口的唯一标识 ID, 由外层传入
	hWnd	指定播放窗口的句柄
返回值	int	
备注	无	

3.3.2 uninitVideoPlay

方法	void uninitVideoPlay();	
简介	initVideoPlay 反向接口, 释放内部资源	
参数	无	
返回值	无	
备注	无	

3.3.3 getIndex

方法	int getIndex();	
简介	获取当前窗口索引	
参数	无	
返回值	当前窗口索引值	
备注	无	

3.3.4 setIndex

方法	void setIndex(int index);	
简介	设置当前窗口索引	

参数	index 索引值
返回值	无
备注	无

3.3.5 setDisplayRegion

方法	bool setDisplayRegion(DISPLAY_RECT *srcRect, bool enable);		
简介	设置或增加显示区域		
参数	srcRect	表示局部显示区域	
	enable	打开(设置)或关闭显示区域	
返回值	成功返回 TRUE，失败返回 FALSE		
备注	可以做局部放大显示		

3.3.6 setPlayerListener

方法	void setPlayerListener(LCOpenSDK_PlayerListener* playerListener);	
简介	设置播放及窗口事件监听	
参数	playerListener	监听指针
返回值	无	
备注	无	

3.3.7 getPlayerListener

方法	(LCOpenSDK_PlayerListener*) getPlayerListener();		
简介	获取播放窗口监听者		
参数	无		
返回值	监听指针		
备注	无		

3.3.8 playAudio

方法	int playAudio();		
简介	伴音音频播放		
参数	无		
返回值	-1	表示失败	
	0	表示成功	
备注	无		

3.3.9 stopAudio

方法	int stopAudio();	
简介	关闭伴音音频	
参数	无	
返回值	-1	表示失败
	0	表示成功
备注	无	

3.3.10 seek

方法	int seek(long seconds);	
简介	录像回放定位功能	
参数	offsetTime 相对录像起始时间的偏移秒数	
返回值	0 表示调用成功，非 0 表示调用失败。是否成功需要根据 onPlayerResult 回调状态码判断，参考业务码定义。	
备注	异步接口	

3.3.11 pause

方法	-(int) pause;	
简介	播放暂停	
参数	无	
返回值	0 表示调用成功，非 0 表示调用失败。是否成功需要根据 onPlayerResult 回调状态码判断，参考业务码定义。	
备注	播放暂停	

3.3.12 resume

方法	int resume();	
简介	恢复录像回放（pause 反向接口）	
参数	无	
返回值	0 表示调用成功，非 0 表示调用失败。是否成功需要根据 onPlayerResult 回调状态码判断，参考业务码定义。	
备注	恢复播放	

3.3.13 rewake

方法	int rewake();	
简介	低功耗设备进入休眠状态过程中唤醒设备	
参数	无	

返回值	0 表示调用成功，非 0 表示调用失败。
备注	恢复播放

3.3.14 snapShot

方法	int snapShot(const char* filePath);	
简介	本地抓图	
参数	filePath	图片文件名（包含路径）
返回值	-1	表示失败
	0	表示成功
备注	播放过程中可调用该接口将一帧画面保存为本地 JPG 格式图片	

3.3.15 startRecord

方法	int startRecord(const char* filePath, int recordType, long long spaceRemain, int convertMode);		
简介	开始本地录制		
参数	filePath	文件路径	
	nRecordType	录制格式(参考 LCOpenSDK_Define E_MEDIA_FORMAT 定义)	
	spaceRemain	磁盘剩余容量（默认为 0xFFFFFFFF）	
	convertMode	录制文件保存模式（默认为 1）	
		0 代表录制成一个文件	
		1 代表当码流分辨率、编码格式、帧率等参数变化会录制成多个文件。	
	0 表示调用成功，非 0 表示调用失败		
备注	播放过程中可调用该接口将视频录制到本地文件		

3.3.16 stopRecord

方法	int stopRecord();	
简介	停止本地录制	
参数	无	
返回值	-1	表示失败
	0	表示成功
备注	与 startRecord 结合使用	

3.3.17 setStreamCallback

方法	void setStreamCallback(OUTPUT_STREAM_FORMAT streamFormat);
简介	设置码流回调
参数	streamFormat 输出的码流格式，目前支持 DAV、PS 和 TS
返回值	无
备注	1、可在播放前或播放过程中调用该接口开启码流回调的功能； 2、码流通过 LCOpenSDK_PlayListener::onStreamCallback 接口导出，每次导出一帧视频或音频，第一帧为视频 I 帧； 3、当不需要流回调功能时，可通过设置输出的码流格式为 OUTPUT_STREAM_FORMAT_NULL 来关闭。

3.3.18 doScale

方法	void doScale(float scale);
简介	设置画面缩放比例
参数	scale 相对缩放比例
返回值	无
备注	

3.3.19 getScale

方法	float getScale();
简介	获取画面缩放比例
参数	无
返回值	相对正常视频画面的缩放比例
备注	

3.3.20 translate

方法	void translate(float x, float y);
简介	窗口平移
参数	x 目的位置点距离屏幕左上角的绝对横坐标 y 目的位置点距离屏幕左上角的绝对纵坐标
返回值	无
备注	无

3.3.21 getTranslateX

方法	float getTranslateX();
简介	获取窗口移动距离（x 坐标）
参数	无
返回值	移动距离
备注	

3.3.22 getTranslateY

方法	float getTranslateY();
简介	获取窗口移动距离（y 坐标）
参数	无
返回值	移动距离
备注	

3.3.23 setMaxScale

方法	void setMaxScale(float maxScale);
简介	设置画面最大缩放比例
参数	maxScale 最大缩放比例
返回值	无
备注	

3.3.24 setIdentity

方法	void setIdentity();
简介	重置画面缩放平移操作
参数	无
返回值	无

备注

3.3.25 **getCurrentStreamMode**

方法	E_STREAM_MODE getCurrentStreamMode();
简介	获取当前拉流类型
参数	无
返回值	当前拉流类型(参考 E_STREAM_MODE 定义)
备注	建议该接口在收到 OnPlayerBegan 回调后调用

3.3.26 **playRtspReal**

方法	int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int channelId, int definitionMode, bool isOpt, const char* playInfo, bool isTryP2P);
简介	预览播放
参数	token: 用户播放 token; deviceId: 设备序列号; decryptKey: 码流解密秘钥; channelID: 通道号; definitionMode: 码流类型, 0: 主码流; 1: 辅码流; isOpt: 是否启用长链接 playInfo: 播放信息; isTryP2P: 是否启用 p2p 拉流
返回值	0/非 0 成功/失败。 实际播放是否成功需要根据 onPlayerResult 回调业务码判断。
备注	无

3.3.27 **playDeviceRecord**

方法	int playDeviceRecord(const char* token, const char* deviceId, const char*
----	--

	<code>decryptKey, int channelId, const char* fileId, int beginTime, int endTime, , int streamType, const char* playInfo, bool isTryP2P);</code>
简介	预览播放
参数	token: 用户播放 token; deviceId: 设备序列号; decryptKey: 码流解密密钥; channelID: 通道号; fileID: 录像文件名, 用于按文件回放录像; beginTime: 录像开始时间, 用于按时间回放; endTime: 录像结束时间, 用于按时间回放; streamType: 主辅码流, 0-主码流 1-辅码流 1, 默认主码流; playInfo: 播放信息; isTryP2P: 是否启用 p2p 拉流
返回值	0/非 0 成功/失败。 实际播放是否成功需要根据 onPlayerResult 回调业务码判断。
备注	无

3.3.28 playCloud

方法	<code>int playCloud(const char* deviceId, int channelId, const char* decryptKey, const char* recordId, int offsetTime, const char* playInfo, const char* signatureUrl, const char* recordPath, const char* recordRegionId);</code>
简介	云录像播放
参数	deviceId: 设备序列号; channelID: 通道号; decryptKey: 码流解密密钥; recordID: 需要播放的文件名; offsetTime: 偏移时间;

	playInfo: 播放信息; needBeginTime: 下载的开始时间; needEndTime: 下载的结束时间; signatureUrl: 云录像查询返回的 m3u 下载地址; recordPath: 存放下载文件的位置; recordRegionId: 云录像查询返回的区域 id;
返回值	0/非 0 成功/失败。 实际播放是否成功需要根据 onPlayerResult 回调业务码判断。
备注	无

3.3.29 stopDeviceRecord

方法	int stopDeviceRecord();
简介	停止录像播放
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	无

3.3.30 stopCloud

方法	int stopCloud();
简介	停止云录像播放
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	无

3.3.31 stopRtspReal

方法	int stopRtspReal();
简介	停止实时播放
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	无

3.3.32 setSEnhanceMode

方法	bool setSEnhanceMode(intmode);
简介	设置去噪模式等级(playAudio 之后调用生效)
参数	无
返回值	True 成功，false 失败
备注	mode(-1,0,1,2,3,4) -1 不进行噪声消除 0 噪声消除程度最低，对有用语音信号的损害最小 4 噪声消除程度最大，对有用语音信号的损害最大

3.3.33 isRecording

方法	bool isRecording();
简介	是否为录制状态
参数	无
返回值	是否正在录制
备注	

3.3.34 isStreamPlayed

方法	bool isStreamPlayed();
----	-------------------------------

简介	是否正在播放
参数	无
返回值	是否正在播放
备注	

3.3.35 getCurrentStreamMode

方法	int getCurrentStreamMode ();
简介	获取当前流模式
参数	无
返回值	E_STREAM_MODE_UNCERTAINTY = -1, //未确定拉流方式 E_STREAM_MODE_P2P = 0, //使用 p2p 拉流 E_STREAM_MODE_MTS = 1, //使用 MTS 拉流
备注	

3.3.36 getPlayerStatus

方法	int getPlayerStatus();
简介	获取播放状态
参数	无
返回值	STATUS_PLAYING 0 STATUS_STOPED 1 STATUS_PAUSE 2 STATUS_REQUESTING 3 STATUS_FAILED 4 STATUS_SEEKING 5 STATUS_ONE_RECORD_FINISHED 6
备注	

3.3.37 setKey

方法	void setKey(const char* passwordKey);	
简介	更新音视频解密密钥	
参数	Passwordkey	秘钥
返回值	无	
备注		

3.3.38 setDecodeEngine

方法	void setDecodeEngine(int engineType);	
简介	设置视频解码引擎	
参数	DECODE_NOTSET = 0	未设置
	DECODE_SW	软解
	DECODE_HW	硬解码拷贝模式(如使用 windows 平台, 使用 d3d9 接口)
	DECODE_HW_FAST	硬解码直接显示模式(如使用 windows 平台, 使用 d3d9 接口)
	DECODE_MSDK	硬解码, 调用 intel media sdk, 已废弃
	DECODE_HW_FAST_D3D11	硬解码直接显示模式,使用 d3d11 接口,仅限 windows 平台
返回值	无	
备注		

3.4 LCOpenSDK_AudioTalk

3.4.1 setTalkerListener

方法	void setTalkerListener(LCOpenSDK_TalkerListener* talkerListener);	
简介	为语音对讲设置监听	

参数	talkerListener	监听指针
返回值	无	
备注	监听者需实现 LCOpenSDK_TalkerListener，可接收语音对讲的状态码回调	

3.4.2 getTalkerListener

方法	LCOpenSDK_TalkerListener* getTalkerListener;	
简介	获取语音对讲监听者	
参数	无	
返回值	监听指针	
备注	无	

3.4.3 playTalk

方法	int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo, bool isTryP2P);	
简介	开始语音对讲	
参数	token: 用户播放 token; deviceId: 设备序列号; channelID: 通道号（设备对讲传-1，通道对讲传对应通道号）; isOpt: 是否启用长链接 decryptKey: 码流解密秘钥; playInfo: 播放信息; isTryP2P: 是否启用 p2p 拉流	
返回值	0/非 0 调用成功/调用失败 实际对讲是否成功需要根据回调状态码判断，回调到监听者对应接口	
备注	1、开启对讲前需设置监听，用于接收业务码回调	

3.4.4 stopTalk

方法	int stopTalk();	
简介	停止语音对讲	

参数	无
返回值	该接口始终返回 0
备注	无

3.4.5 playSound

方法	int playSound();
简介	开启对讲音播放
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	打开对讲音

3.4.6 stopSound

方法	int stopSound();
简介	关闭对讲音
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	关闭对讲音

3.4.7 startSampleAudio

方法	int startSampleAudio();
简介	开始音频采集
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	该接口需要在对讲成功的回调上来后调用才有效

3.4.8 stopSampleAudio

方法	NSInteger stopSampleAudio();
----	-------------------------------------

简介	停止音频采集
参数	无
返回值	0 表示调用成功，非 0 表示调用失败。
备注	在停止对讲前需要调用该接口

3.5 LCOpenSDK_Utils

3.5.1 getSaveStreamPath

方法	<code>const char* getSaveStreamPath();</code>
简介	获取码流文件路径
参数	无
返回值	文件路径
备注	无

3.5.2 computeMD5Lower

方法	<code>Void computeMD5Lower(const char* srcStr, char outStr[128]);</code>
简介	计算 MD5 加密值，结果小写
参数	srcStr 加密值 srcStr 加密后的输出值
返回值	无
备注	无

3.5.3 computeMD5Upper

方法	<code>void computeMD5Upper (const char* srcStr, char outStr[128]);</code>
简介	计算 MD5 加密值，结果大写
参数	srcStr 加密值 srcStr 加密后值

返回值	无
备注	无

3.5.4 hmac_sha512

方法	char * hmac_sha512(const char * key, const char * data, char outData[128]);		
简介	HMAC-SHA512 加密		
参数	key	加密密钥	
	data	加密数据	
	outData	加密后值	
返回值	无		
备注	无		

3.5.5 sha512Cry

方法	char * sha512Cry(const char * data);		
简介	SHA512 加密		
参数	data	加密数据	
返回值	加密后数据		
备注	该接口内部会 new 出字符数组，返回的值使用完之后记得及时释放		

3.5.6 compute1

方法	void compute1(const char* srcStr, char outStr[128]);		
简介	Base64 MD5 小写加密		
参数	srcStr	待加密数据	
	outStr	加密后数据	
返回值	无		
备注	无		

3.5.7 compute2

方法	void compute2(const char* srcStr, char outStr[128]);	
简介	Base64 MD5 大写加密	
参数	srcStr	待加密数据
	outStr	加密后数据
返回值	无	
备注	无	

3.5.8 getMac

方法	void getMac(char outMac[128]);	
简介	获取设备的 mac 地址	
参数	outMac	作为输入，可以传空字符数组
返回值	outMac	输出的 mac 地址
备注	无	

3.5.9 getOsVersion

方法	void getOsVersion(char version[128]);	
简介	获取 OS 版本信息	
参数	version	作为输入，可以传空字符数组
返回值	version	输出的 os 的版本信息
备注	无	

3.5.10 loadSubModules

方法	void loadSubModules();
----	-------------------------------

简介	加载媒体子库
参数	无
返回值	无
备注	进行业务之前必须先加载媒体子库

3.5.11 setMobileLogLevel

方法	void SetMobileLogLevel(OpenSDK_LogLevel level = All);
简介	设置日志输出等级
参数	logLevel 输出日志等级(参考 OpenSDK_LogLevel)
返回值	无
备注	建议该接口在其他所有接口前调用

3.5.12 setSaveStreamFlag

方法	void setSaveStreamFlag(bool isOpen);
简介	设置是否开启码流写文件(播放、对讲)
参数	isOpen YES/NO 开启/关闭
返回值	无
备注	在开始播放(对讲)前调用才有效，发布版本请不要调用该接口，仅调试时使用

3.6 LCOpenSDK_PlayerListener(播放业务及窗口事件回调)

上层如需接收播放业务及窗口事件的回调，需遵守 LCOpenSDK_PlayerListener 协议，通过 setPlayerListener 接口设置监听，即能接收相关回调。

3.6.1 onPlayerResult

方法	void onPlayerResult(int index, const char* code, int resultSource);	
简介	视频播放的状态码回调	
参数	index	播放窗口的唯一标识 ID
	code	状态码
	type	业务类型
返回值	无	
备注	Index: 窗口索引; code: 根据 type 不同而不同, 参见 错误!未找到引用源。E_RESULT_PROTO_TYPE_DHHTTP_STATE E_RESULT_PROTO_TYPE_FILE_STATE type: 参见 E_RESULT_PROTO_TYPE ; 接收到错误回调后需关闭会话, 具体判断和处理可参考 Demo	

3.6.2 onResolutionChanged

方法	void onResolutionChanged(int index, int width, int height)	
简介	分辨率改变的回调	
参数	Index	播放窗口的唯一标识 ID
	width	宽
	height	高
返回值	无	
备注	无	

3.6.3 onReceiveData

方法	void onReceiveData(int index, int len);	
简介	接收码流数据长度的回调	
参数	index	播放窗口的唯一标识 ID
	len	数据长度
返回值	无	
备注	可根据 len 计算码流速率。	

3.6.4 on PlayBegan

方法	void onPlayBegan(int index);
----	-------------------------------------

简介	开始播放(第一帧已经解码完成)	
参数	index	窗口序号
返回值	无	
备注	开始播放则会触发此回调事件(包括暂停后播放、seek 后播放)。	

3. 6. 5 onPlayFinished

方法	void onPlayFinished (int index);	
简介	录像播放结束	
参数	index	窗口序号
返回值	无	
备注	设备录像或本地录像回放结束触发此回调事件。	

3. 6. 6 onPlayerTime

方法	void onPlayerTime(int index, long time);	
简介	播放时间回调	
参数	index	窗口序列号
	time	UTC 时间
返回值	无	
备注	当前解码时戳回调。	

3. 6. 7 onBadFile

方法	void onBadFile(const char* context);	
简介	播放本地录像文件异常	
参数	context,	上下文
返回值	无	
备注	播放本地文件时检测到为无效文件	

3. 6. 8 onFrameLost

方法	void onFrameLost(const char* context);
简介	丢帧回调
参数	context, 上下文
返回值	无
备注	无

3. 6. 9 onRecordStop

方法	void onRecordStop(const char* context, int error);
简介	录制结束回调
参数	error 错误代码(0 表示正常结束) index, 窗口序号 context, 上下文
返回值	无

3. 6. 10 onFileTime

方法	void onFileTime(const char* context, long beginTime, long endTime);
简介	本地文件开始时间和结束时间
参数	beginTime 开始时间 endTime 结束时间 context 上下文
返回值	无

3. 6. 11 onPlayerTimeAndStamp

方法	void onPlayerTimeAndStamp(const char* context, long time, long stampTime);
----	---

简介	视频解码时间戳的回调	
参数	time	UTC 时间
	stampTime	时间戳
	context	上下文
返回值	无	
备注	本地 mp4 文件播放时，time 为已播放的秒数，其他为 UTC 时间	

3.6.12 onAudioTimeAndStamp

方法	void onAudioTimeAndStamp(const char* context, long time, long stampTime, bool isDetach);	
简介	音频解码时间戳的回调	
参数	time	UTC 时间
	stampTime	时间戳
	context	上下文
	isDetach	音频是否分离
返回值	无	
备注		

3.6.13 onStreamLogInfo

方法	void onStreamLogInfo(const char* context, const char* message, unsigned int length);	
简介	回调拉流日志信息	
参数	Context	上下文
	Message	日志内容
	length	长度
返回值	无	
备注	无	

3.7 LCOpenSDK_TalkerListener

3.7.1 onTalkPlayReady

方法	void onTalkPlayReady();	
简介	语音对讲成功的状态码回调	
参数	无	
返回值	无	
备注		

3.7.2 onTalkResult

方法	void onTalkResult(const char* error, int type);	
简介	语音对讲的状态码回调	
参数	error	状态码
	type	业务类型
返回值	无	
备注	error: 根据 type 不同而不同, 参见 3.2.2.1 E_RESULT_PROTO_TYPE_RTSP_STATE 3.2.2.1 E_RESULT_PROTO_TYPE_DHHTTP_STATE type: 参见 3.2.2.1 E_RESULT_PROTO_TYPE 接收到错误回调后需关闭会话, 具体判断和处理可参考 Demo	

3.8 LCOpenSDK_Download

3.8.1 startDownload

方法	int startDownload(int index, const char* token, const char* filepath, const char* deviceId, const char* decryptKey, const char* fileId, int needBeginTime, int needEndTime, const char* playInfo, int channelId);
简介	设备录像下载
参数	index: 下载的唯一标识 ID; token: 用户播放 token; filepath: 存放下载文件的位置; deviceId: 设备序列号; decryptKey: 码流解密密钥; fileId: 需要下载的文件名; needBeginTime: 下载的开始时间; needEndTime: 下载的结束时间; playInfo: 播放信息; channelId: 通道号;
返回值	0/非 0 成功/失败。 实际下载是否成功需要根据 onPlayerResult 回调业务码判断。

备注	无
----	---

3.8.2 startCloudDownload

方法	<code>int startCloudDownload(int index, const char* filepath, const char* deviceId, int channelId, const char* recordId, int needBeginTime, int needEndTime, int offsetTime, const char* playInfo, const char* signatureUrl, const char* recordPath, const char* recordRegionId);</code>
简介	云录像下载
参数	index: 下载的唯一标识 ID; filepath: 存放下载文件的位置; deviceId: 设备序列号; channelId: 通道号; recordId: 需要下载的文件名; needBeginTime: 下载的开始时间; needEndTime: 下载的结束时间; offsetTime: 偏移时间; playInfo: 播放信息; signatureUrl: 云录像查询返回的 m3u 下载地址; recordPath: 存放下载文件的位置; recordRegionId: 云录像查询返回的区域 id;
返回值	0/非 0 成功/失败。 实际下载是否成功需要根据 onPlayerResult 回调业务码判断。
备注	无

3.8.3 stopDownload

方法	<code>int stopDownload(int index);</code>
简介	取消下载
参数	index: 下载的标识 ID;

返回值	0 表示调用成功，非 0 表示调用失败。
备注	无

3.9 LCOpenSDK_DownloadListener

3.9.1 onDownloadReceiveData

方法	void onDownloadReceiveData(int index, int datalen);	
简介	接收码流数据长度的回调	
参数	index	下载的唯一标识 ID
	len	数据长度
返回值	无	
备注	可根据 len 计算码流速率。	

3.9.2 onDownloadState

方法	void onDownloadState(int index, const char* code, int type);	
简介	下载状态回调	
参数	index	下载的唯一标识 ID
	code	状态码（可参考 3.2.2 ）
	type	协议类型（可参考 3.2.2 ）
返回值	无	
备注		

3.9.3 onDownloadProgress

方法	void onDownloadProgress(int index, float progress, int type);	
简介	下载进度回调	
参数	index	下载的唯一标识 ID
	progress	下载进度
	type	协议类型（可参考 3.2.2 ）
返回值	无	
备注		

4 SDK 对接使用说明

PC OpenSDK Demo 中已经实现了目前提供的所有核心功能，开发者如果理解了 Demo 中

源代码的实现逻辑，那么把这些功能移植到自己的应用中将会比较简单。

PC Demo 工程源码请见 PCOpenDemo 目录，工程主要引用 PC SDK,并且围绕 SDK 实现基于视频的核心功能。可以借鉴提供的 demo 开发自己的客户端。

4.1 SDK 对接流程和关键业务接口调用

PC OpenSDK 集成的主要功能包含：实时预览、高标清切换、本地录制、抓图、云台控制、语音对讲、设备录像回放功能。

4.1.1 SDK 对接流程

PC SDK 是 visual studio2005 编译的，如果自己的 PC 没有相关环境，需要将 IoTApplication_VideoPlay_PC_C_Plus_Plus_V1.1.2\depend\64bit_lib\other 里面的.manifest 文件和 vc 运行时库加载进项目（将目录下面的.manifest 和关于 vc 的运行时库拷贝到程序运行目录下）。

所需的开发环境在第二章已经描述，下面是一些初始化接口调用的顺序和解释（其中的截图是 demo 中代码的示例）：

(1) void LCOpenSdk_Utils::loadSubModules();

加载拉流等媒体子库，确保拉流的正常进行；

```
LCOpenSdk_Utils::loadSubModules();
```

(2) void SetMobileLogFile(const char* fileName = "LCOpenSDK", int maxSize = 10*1024*1024, OpenSdk_LogFileMode fileMode = NewFile);

设置 SDK 日志接口；

```
LCOpenSdk_Utils::SetLogInfo(LCOpenSdk_Utils::A11, "DEMO", 10*1024*1024, LCOpenSdk_Utils::RollFile);
```

(3) void initOpenApi(const char* host, int port, const char* caPath);

初始化协议库，其中的参数 host 和 port 分别是开发者平台注册账号时拿到的 host，port 默认是 443 端口，caPath 是 SDK 证书路径。

```
m_OpenApiHandle->initOpenApi(param.host.toLatin1().data(), param.port, NULL);
```

4.1.2 关键业务接口调用

首先获取设备列表接口所需的 token：需要访问接口（具体该接口的请求参数和返回参数参考大华云开发者平台）/open-api/api-base/auth/getAppAccessToken（具体使用可以参考 demo 中” int TokenCtrl::getIOTToken(QString host, OUT QString& token, QString AccessKey)”函数）。

其次获取设备列表同样该接口的请求参数和返回参数参考大华云开发者平台 /open-api/api-iot/device/getDeviceList（可以参考 demo 中 “void TokenCtrl::getIOTPlayInfo()”函数）。

4.1.2.1 实时预览

首先在播放之前进行初始化，需要调用 int initVideoPlay(int index, void* hWnd)以及设置监听回调 void setPlayerListener(LCOpenSDK_PlayerListener* playerListener)（接口的参数解释在第三章已经表述，具体用法可以参考 demo 中 int PlayCtrl::init(QWidget *videoBase)函数）

```
int PlayCtrl::init(QWidget *videoBase)
{
    int ret = 0;
    do
    {
        if (NULL == videoBase)
        {
            ret = -1;
            break;
        }

        m_videoPlay = new LCOpenSDK_VideoPlay();
        if (NULL == m_videoPlay)
        {
            ret = -1;
            break;
        }

        if (0 != m_videoPlay->initVideoPlay(0, (void*)(videoBase->winId())))
        {
            DLOG("LCOpenSDK_VideoPlay init fail!!!\n");
            ret = -2;
            break;
        }
        else
        {
            m_videoPlay->setPlayerListener(this);
        }
    }
}
```

int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int channelId, int definitionMode, bool isOpt, const char* playInfo);

实时预览接口参数的解释在第三章已经表述，这里需要注意的是 playInfo 参数是从开发

者平台/open-api/api-iot/device/getDeviceList 接口获取的，获取到的 playInfo 是加密的，需要调用 SDK 的 LCOpenSDK_Crypter::decryptIOTPlayInfo 的解密接口进行解密后传入 playRtspReal 函数。播放的成功和失败可以在 void onPlayerResult(int index, const char* code, int resultSource) 的回调中查看具体原因（可参考 demo 中的 PlayCtrl 类中实现的回调函数，它继承自 LCOpenSDK_PlayerListener）

```
class PlayCtrl : public QObject, public LCOpenSDK_PlayerListener
```

4.1.2.2 高标清切换

```
int playRtspReal(const char* token, const char* deviceId, const char* decryptKey, int  
channellID, int definitionMode, bool isOpt, const char* playInfo);
```

该函数中 definitionMode 参数：0 代表高清，1 代表标清

```
ret = m_videoPlay->playRtspReal(token.c_str(), m_iotDeviceList->deviceId.c_str(),  
m_iotDeviceList->decryptKey.c_str(), channID, definitionMode, false, playInfo.c_str());
```

4.1.2.3 本地录制

```
int startRecord(const char* filePath, int recordType, long long spaceRemain = 0xFFFFFFFF, int  
convertMode = 1);
```

该接口是录制实时预览画面，可以参考 demo 中的调用

```
ret = m_videoPlay->startRecord(getRecordPath().toLocal8Bit().data(), 1);
```

4.1.2.4 抓图

```
int snapShot(const char* filePath);
```

该接口是实时预览的时候进行进行视频的截图，可以参考 demo，直接传入一个保存图片的路径即可

```
ret = m_videoPlay->snapShot(getSnapShotPath().toLocal8Bit().data());
```

4.1.2.5 云台控制

该功能用到的是 OpenApiClient 协议库，对应的头文件在 depend / include /

LCOpenApiClient / LCOpenApi 中的 ControlPTZ.h, 各个参数的传递可以参考 demo 中的实现

```
int OpenApiCtrl::controlPTZ(const PTZ_CONTROL_PARAMETERS &parameters)
{
    ControlPTZRequest *req;
    ControlPTZResponse *resp;

    int ret = 0;
    char iCh[10]={0};
    char strDuration[10] = {0};
    _snprintf(iCh, sizeof(iCh), "%d", parameters.channelId);
    _snprintf(strDuration, sizeof(strDuration), "%d", parameters.duration);
    req = LCOOPENAPI_INIT(ControlPTZRequest);
    resp = LCOOPENAPI_INIT(ControlPTZResponse);
    req->data.deviceId = CS(parameters.deviceId.toString().c_str());
    req->data.duration = CS(strDuration);
    req->data.channelId = CS(iCh);
    req->data.h = parameters.horizon;
    req->data.v = parameters.vertical;
    req->data.z = parameters.zoom;
    req->data.operation = CS(parameters.operate.toString().c_str());

    req->data.token = CS(userToken.c_str());

    ret = m_OpenApiHandle->request(req, resp, 10, 0, parameters.playInfo.c_str());
}
```

4.1.2.6 语音对讲

两个概念:

(1) 伴音: 和视频路一起过来的声音, 可以当做设备过来的声音。通过 playAudio()和 stopAudio()接口来打开和关闭设备过来的声音(如果设备没有开启伴音, 尝试通过开发者平台接口/open-api/api-iot/device/setAbilityStatus 开启 Audio Encode Control 使能);

(2) 对讲音: 通过 playTalk 接口开启。双方对讲可以直接通过该接口进行。

两种方法开启双边对讲:

把伴音当做对讲音。首先视频渲染成功之后, 调用 playAudio()来打开设备过来的声音; 然后开启对讲, 调用 int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo)接口, 待 onTalkResult 回调对讲成功之后 (不要在回调中进行业务, 可另外开启一个线程) 进行音频的采集 startSampleAudio();

```

void RealPlayCtrl::onTalkResult(const char* error, int type)
{
    DLOG("onTalkResult-%s-%d\n", error, type);
    if (!isNumber(error))
    {
        DLOG("business error[%s]!!!\n", error);
        emit sgnTalkerStatusChanged(STATUS_FAIL);
        return;
    }

    int nError = atoi(error);
    if(0 == type){
        if (STATE_RTSP_DESCRIBE_READY == nError)
        {
            DLOG("talk ready\n");
            return;
        }
        else if (STATE_RTSP_PLAY_READY == nError)
        {
            // start sample audio
            m_thread = std::thread(&RealPlayCtrl::audioSampletreadFunc, this);
            m_thread.detach();
            DLOG("start to talk\n");
            m_isTalking = true;
        }
    }
}

```

int playTalk(const char* token, const char* deviceId, int channelId, const char* decryptKey, bool isOpt, const char* playInfo);

不把伴音当做对讲音。首先视频渲染成功之后，不调用 playAudio()，意思是不去开启伴音。只去调用 playTalk 接口开启对讲，待 onTalkResult 回调对讲成功之后（需在另外一个线程开启采集）进行音频的采集 startSampleAudio()。此时需要调用 playSound()接口打开设备过来的声音，关闭对讲的时候记得调用 stopSound()。

4.1.2.7 设备录像回放

int playDeviceRecord(const char* token, const char* deviceId, const char* decryptKey, int channelId, const char* fileId, int beginTime, int endTime, int offsetTime, bool isOpt, const char* playInfo);

该接口可以用作按时间或者按文件播放均可。

按文件播放只需要传入查询到的 fileId 传入即可，beginTime 和 endTime 传 0 或者传其他值都可；按时间播放，需要将 fileId 传空字符串 ""，beginTime 和 endTime 传你想要播的录像片段的时间。

```

ret = m_videoPlay->playDeviceRecord(token.c_str(), m_recordInfo->deviceId.toStdString().c_str(), m_iotDeviceList->decryptKey.c_str(),
    m_recordInfo->channelId, m_recordInfo->name.toStdString().c_str(), beginTime.toTime_t(), endTime.toTime_t(), 0, false, playInfo.c_str());

```

其中需要注意的是录像文件信息的获取，查询录像信息的接口和云台接口在同一文件夹中，接口文件 QueryLocalRecords.h，具体使用可以参考 demo 代码。

```

int RecordListCtrl::getRecordInfo(const QString &deviceId, int channelId, std::string playInfo, const QDate &date)
{
    clearRecordList();

    // QString beginTime = "";
    // QString endTime = "";
    // getBeginTimeAndEndTime(date, beginTime, endTime);
    QString beginTime = date.toString("yyyy-MM-dd").append(" 00:00:00");
    QString endTime = date.toString("yyyy-MM-dd").append(" 23:59:59");

    RECORD_PARAMETERS parameters;
    parameters.deviceId = deviceId;
    parameters.channelId = channelId;
    parameters.beginTime = beginTime;
    parameters.endTime = endTime;
    parameters.playInfo = playInfo;

    int totalRecNum = 0;
    if (DEVICE_RECORD == m_recordType)
    {
        totalRecNum = OpenApiCtrl::getInstance()->getDeviceRecordNum(parameters);
    }
}

```

```

int OpenApiCtrl::getDeviceRecordNum(const RECORD_PARAMETERS &parameters)
{
    if (!m_IsGetTokenOK)
    {
        DLOG("get Token Failed!!!\n");
        return RET_FAIL;
    }

    QueryLocalRecordNumRequest *req;
    QueryLocalRecordNumResponse *resp;
    int ret = 0;
    char strCh[10] = {0};
    _snprintf(strCh, sizeof(strCh), "%d", parameters.channelId);
    req = LCOPENAPI_INIT(QueryLocalRecordNumRequest);
    resp = LCOPENAPI_INIT(QueryLocalRecordNumResponse);

    std::string userToken = "";
    std::vector<DEVICE_INFO> deviceVec = m_tokenCtrl->getDeviceVec();
    for (int i = 0; i < deviceVec.size(); i++)
    {
        // Json::Reader reader;
        // Json::Value root;
        // QJsonParseError jsonError;
        // QJsonDocument json = QJsonDocument::fromJson(deviceVec.at(i).playInfo.c_str(), &jsonError);
        if (jsonError.error != QJsonParseError::NoError)
        {
            QString strError = jsonError.errorString();
            continue;
        }

        if (!json.isObject())
        {
            continue;
        }

        userToken = json.object().value("userToken").toString().toStdString();
        //if (reader.parse(m_tokenCtrl.deviceVec.at(i).playInfo, root)) {
        // userToken = root["userToken"].asString();
        // }
    }
}

```

4.1.2.8 设备录像下载

int startDownload(int index, const char* token, const char* filepath, const char* deviceId, const char* decryptKey, const char* fileId, int beginTime, int endTime, const char* playInfo, int channelId);

该接口可以用作按时间或者按文件下载均可。

按文件下载需要传入查询到的 fileID、beginTime 和 endTime(其中的 beginTime 和 endTime 在内部做下载的总文件时长使用);

按时间下载, 需要将 fileID 传空字符串 "", beginTime 和 endTime 传你想要播的录像片段的时间(需要注意的是 beginTime 和 endTime 需要在某一个文件中间, 这两个时间也是在内部做下载的总文件时长使用)。

下面以按文件下载举例, 具体参考 demo 中实现

```
QDateTime beginTime = QDateTime::fromString(record_info->beginTime, "yyyy-MM-dd HH:mm:ss");
QDateTime endTime = QDateTime::fromString(record_info->endTime, "yyyy-MM-dd HH:mm:ss");
// playInfo
std::string playInfo = iotDeviceList->playInfo;
m_downloadingFilePath = QString(m_deviceRecordsDir + "/" + record_info->deviceId + "_download_" + strBeginTime + ".mp4").toLatin1();
m_lcopensdkDownloadInstance->startDownload(index, record_info->token.toString().c_str(), m_downloadingFilePath.data(), iotDeviceList->deviceId.c_str(),
iotDeviceList->decryptKey.c_str(), record_info->name.toString().c_str(), beginTime.toTime_t(), endTime.toTime_t(), playInfo.c_str(), 0);
```

4.1.2.9 ARC 对讲

目前 OpenSDK 支持双向对讲功能, 开启对讲之前需要提前申请音频采集权限。ARC 对讲功能对接流程如下:

调用开发者平台接口 (/open-api/api-converter/device/listDeviceDetailsByIds) 获取设备列表, 平台返回的数据包含设备序列号和 playInfo (需要解密, 参考 4.1.2.1 中文档描述)) 信息。开发者需要关注这两个数据, 后续开启对讲需要用到这两个参数。

使用 SDK 的 LCOpenSDK_AudioTalk 类的 playTalk() 接口可以开启语音对讲。开启对讲时, 对讲参数类 TalkParams 中的 deviceId、playInfo 值已经在获取设备列表时得到了; channelId 的值由平台推送的消息中得到。其他的均可参考 4.1.2.6 章节。

4.1.2.10 启用 p2p

SDK 中涉及到启用 p2p 功能的接口有 playRtspReal、playTalk、playDeviceRecord 接口, 需要在原有的播放流程中, 增加初始化和预打洞步骤。

1. app 开始登录时, 通过开发者平台获取 p2p 服务的接口 (/open-api/api-biz/p2p/info) 获取 p2p 服务信息, 并通过 SDK 的 configureP2PServers 接口初始化 P2P 服务信息。
2. 获取设备列表时, 需要将设备的 playInfo 信息保存下来, 并通过 SDK 的 addDevices 接口开始预打洞。
3. 在拉流时, 需要将 isTryP2P 参数设置为 true。SDK 内部会使用 p2p 开始拉流, 如果 p2p 打洞失败, SDK 内部会使用 MTS 拉流。

拉流成功后, 可以通过 LCOpenSDK_VideoPlay 的 getCurrentStreamMode() 函数获取本次拉流的类型, 如果是 0 (E_STREAM_MODE_P2P), 说明此次拉流使用的是 p2p 拉流。

4.1.2.11 低功耗设备

SDK 支持低功耗设备的实时预览，设备休眠时能直接唤醒并拉流，实时预览时，设备休眠进入会有回调提醒，并且能打断设备休眠。

`onPlayerResult()` 回调的状态码以 80 开头，后面的两位数为休眠倒计时秒数，例如 8010 表示还有 10 秒钟休眠。设备进入休眠状态时会抛出“603038”状态码。如果想在即将进入休眠的期间打断休眠，可以调用 `rewake()` 接口。

Demo 获取设备列表后，如果设备处于休眠状态会在列表项显示设备 `sleep`，点击实时预览，开始对休眠设备拉流，休眠的设备在拉流时会被自动唤醒。

实时预览一段时间后，设备准备进入休眠，此时 Demo 会检测到 SDK 上抛的状态码，并在实时预览页面显示倒计时秒数，如果点击实时预览页面，Demo 会调用打断休眠接口，继续播放；如果不管，会在倒计时结束后停止拉流并退出。

4.2 Demo 功能演示说明

4.2.1 背景

为了让开发者快速掌握如何对接开发者平台 PC SDK，以及对 SDK 的功能有一个全面的了解，利用 PC SDK 开发了 PC Demo 应用，仅供参考。

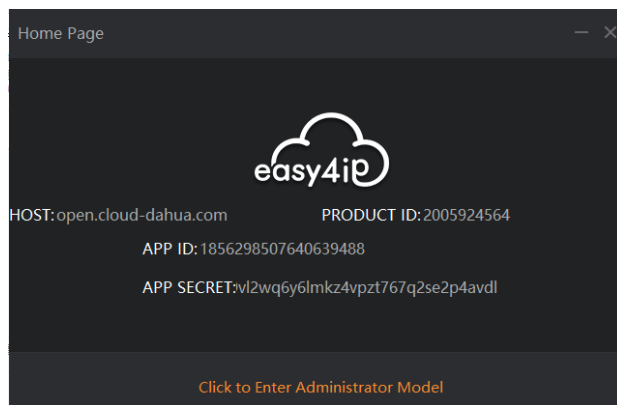
PC Demo 工程源码请见 CPPDemo 压缩包，工程主要引用 PC SDK，并且围绕 SDK 实现基于视频的核心功能。可以借鉴提供的 demo 开发自己的客户端。

4.2.2 功能介绍和使用说明

如果前期仅想了解提供的 DEMO 实现的主要功能，可以直接阅读本文档了解 demo 的功能。后期建议开发者将 demo 工程部署到自己的开发环境 IDE 中，并且编译通过后进行真机调试，结合本文档亲身体会 demo 中的功能。

4.2.2.1 账号登录

打开 SDK demo 的软件，如下图所示：

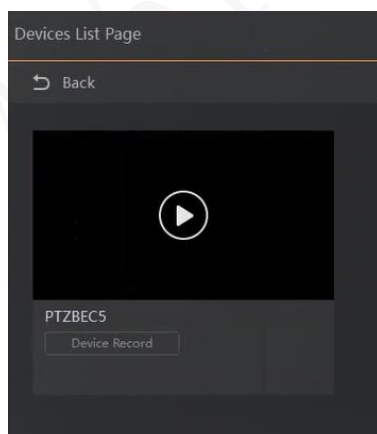


其中 HOST 为环境域名，App ID、App Secret 及 ProductId 可以在开发者平台的产品概况中查询，前提是需要先在平台 <https://open.cloud-dahua.com/> 中完成产品创建。



填写完以上信息，点击登录，即可登录进入设备列表页面。

4.2.2.2 获取设备列表

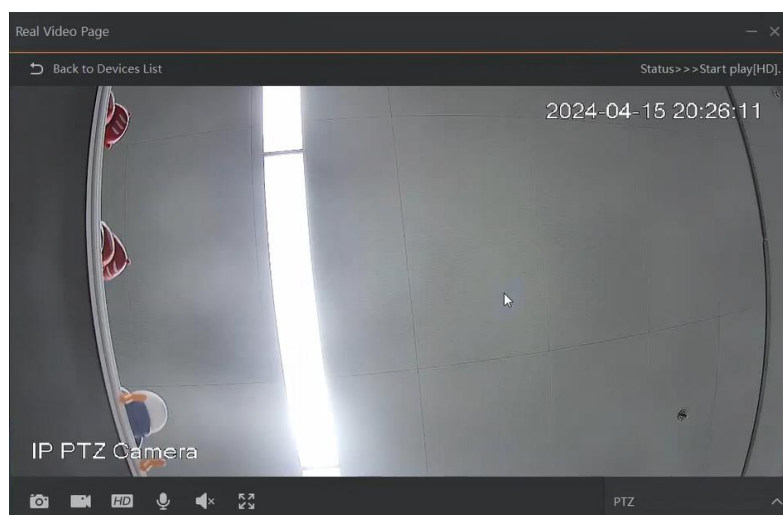


点击进入设备列表之后需要向开放平台获取本账号下的设备列表。

4.2.2.3 实时预览和对讲

在主界面点击摄像头的实时按钮，出现如下窗口，即可播放摄像头的视频。

窗口里会显示本地抓图、录像、清晰度的切换、语音对讲的开关、音频的开关、全屏开关和云台控制。



4.2.2.4 云台控制

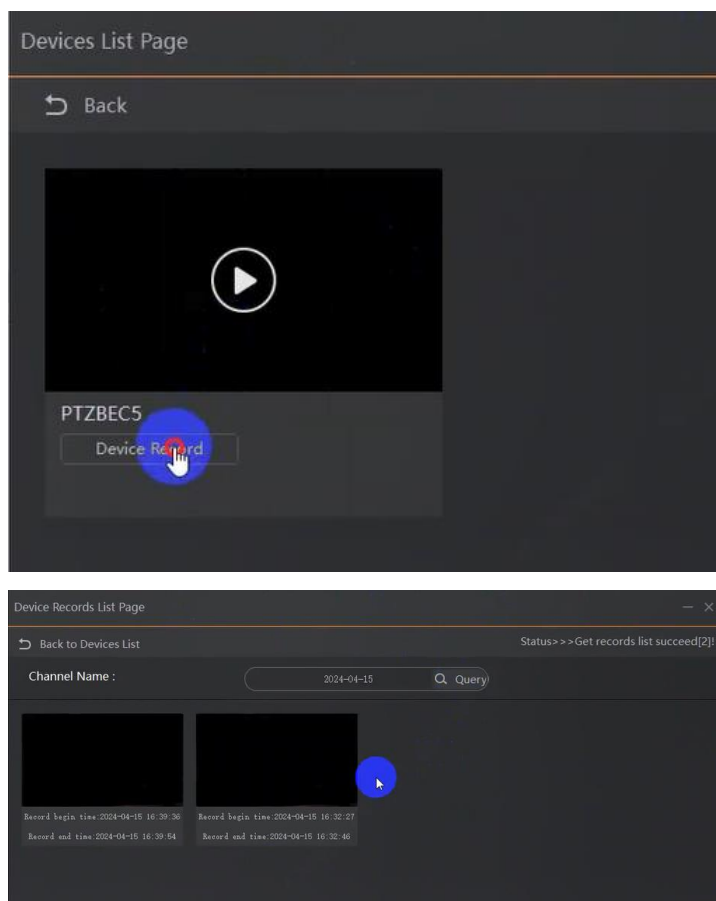


在视频播放页面，点击“PTZ”按钮展开，然后选择控制摄像头的方向，即可实现云台的上下左右转动。

注：云台控制功能仅针对云台设备。

4.2.2.5 设备录像播放

点击设备录像按钮，出现如下界面。



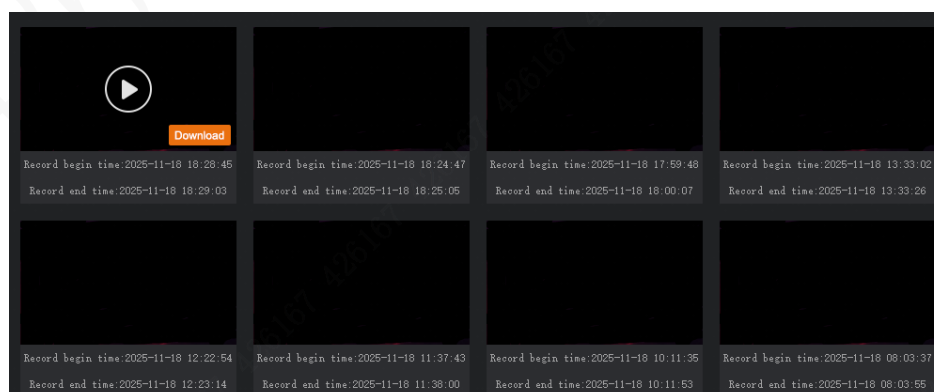
注：Demo 中默认只查询 24 小时内的最近 10 条录像，开发者可根据自身应用的业务需要自行修改查询条件。

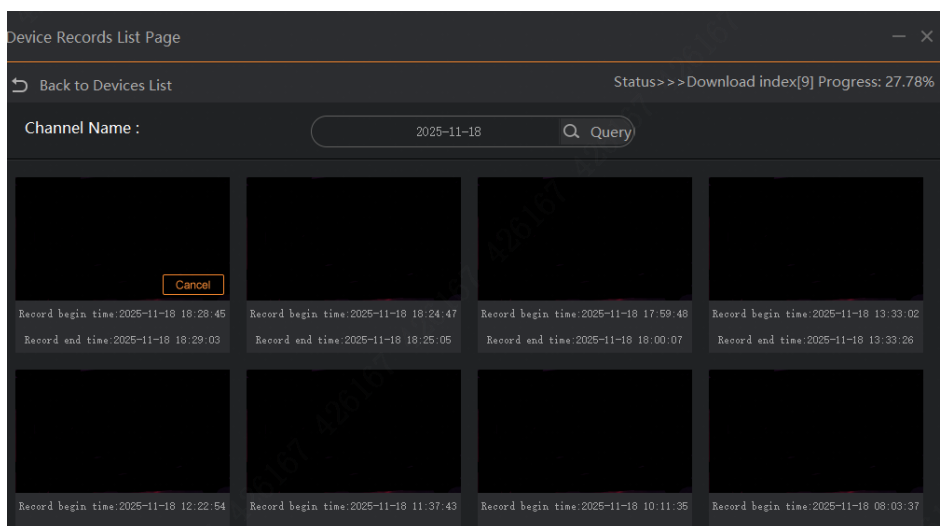
点击任意一个录像跳转至录像播放界面。

4.2.2.6 设备录像下载

在设备录像界面上，鼠标停留在某一文件内，即显示“Download”，点击即可下载。

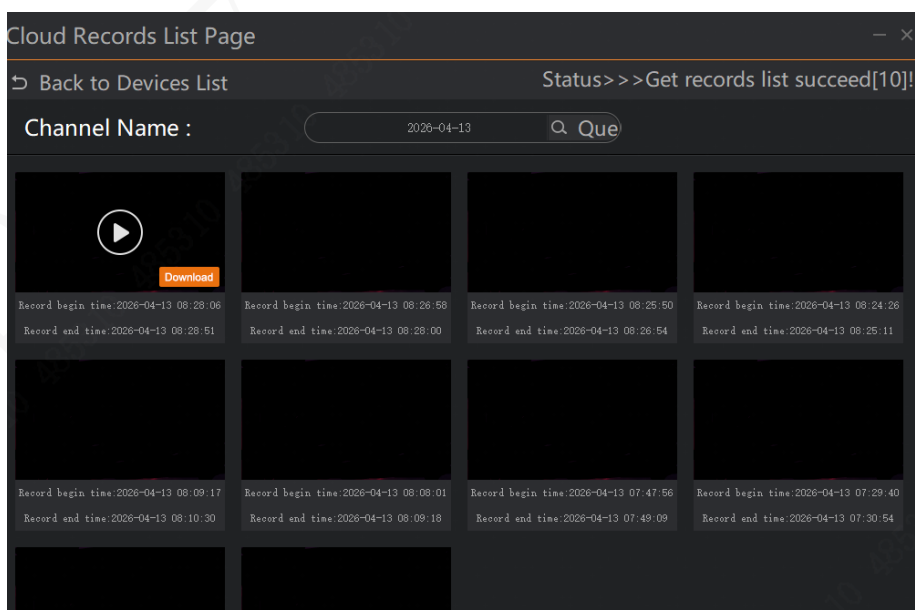
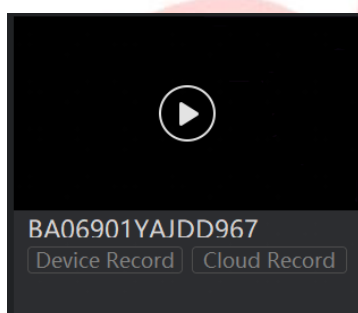
下载过程中，右上角可显示下载进度。

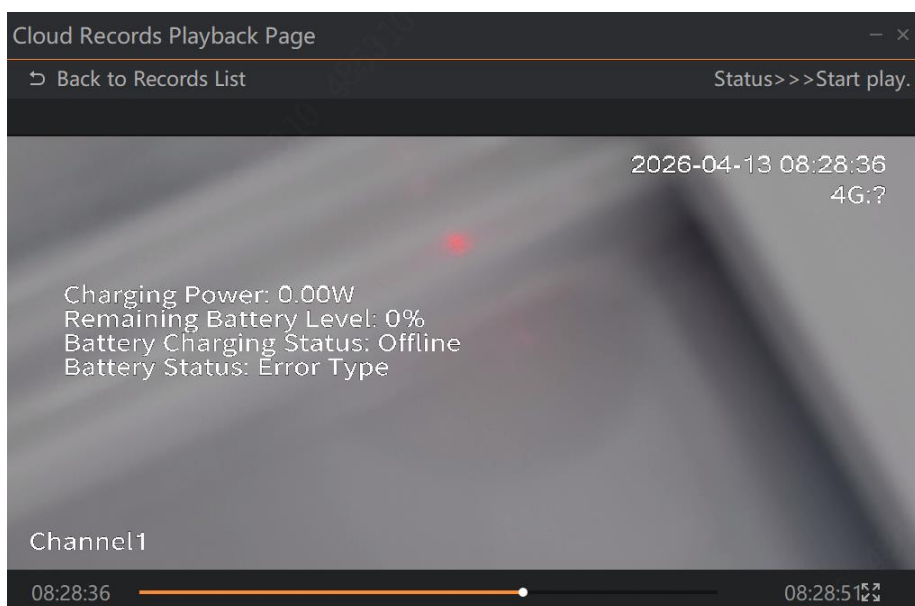




4.2.2.7 云录像播放

点击录像界面上“Cloud Record”，即可进入云录像界面，点击文件的中间播放按钮即可进行云录像回放





4.2.2.8 云录像下载

在云录像界面上，鼠标停留在某一文件内，即显示“Download”，点击即可下载。下载过程中，右上角可显示下载进度。

